LAPORAN JOBSHEET
“13- Middleware & Route Protection”
MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

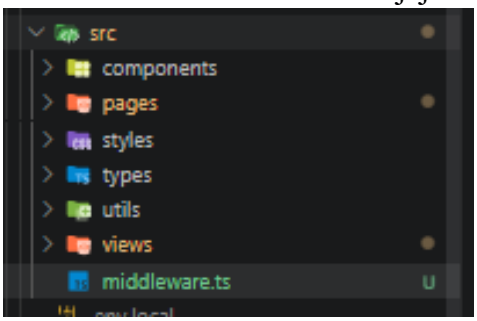
2026

Bagian 1 – Membuat Middleware

1. Modifikasi file index.tsx pada folder src/pages/product

```
P2 > Praktikum > next-routing > src > pages > product > index.tsx > ProductPage
1   import { useRouter } from "next/router";
2   import { useEffect, useState } from "react";
3   import HeroSection from "../../views/product/components/HeroSection";
4   import MainSection from "../../views/product/components/MainSection";
5   import ShowProducts from "../../views/product";
6   import useSWR from "swr";
7   import fetcher from "../../utils/swr/fetcher";
8   import Head from "next/head";
9
10  // const fetcher = (url: string) => fetch(url).then((res) => res.json());
11
12  const ProductPage = () => {
13    // const [isLogin, setIsLogin] = useState(false);
14    const { push } = useRouter();
15    const [isRefreshing, setIsRefreshing] = useState(false);
16
17    const { data, error, isLoading, isValidating, mutate } = useSWR(
18      "/api/product",
19      fetcher,
20    );
21 >  //cek apakah data, error, dan isLoading sudah benar···
29
30    const handleLogout = () => {
31      localStorage.removeItem("isLogin");
32      push("/auth/login");
33    };
34
35    const handleRefresh = () => {
36      setIsRefreshing(true);
37      mutate().finally(() => {
38        setIsRefreshing(false);
39      });
40    };
41
42    return (
43      <>
44        <Head>
45          <title>Halaman Produk | My App</title>
46        </Head>
47        <div className="min-h-screen bg-gray-100">
48          <HeroSection />
```

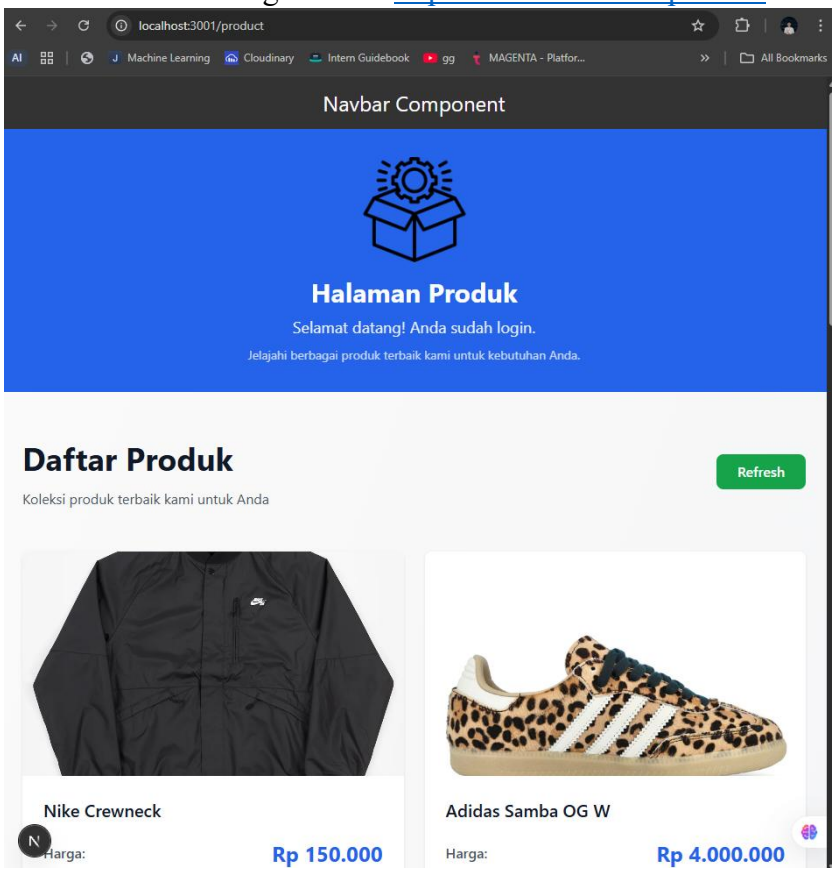
2. Buat file: src/middleware.ts Sejajar dengan folder pages.



Bagian 2 – Struktur Dasar Middleware

```
index.tsx 3, M ×   middleware.ts 1, U ×
P2 > Praktikum > next-routing > src > middleware.ts > middleware
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
    return NextResponse.next();
}
```

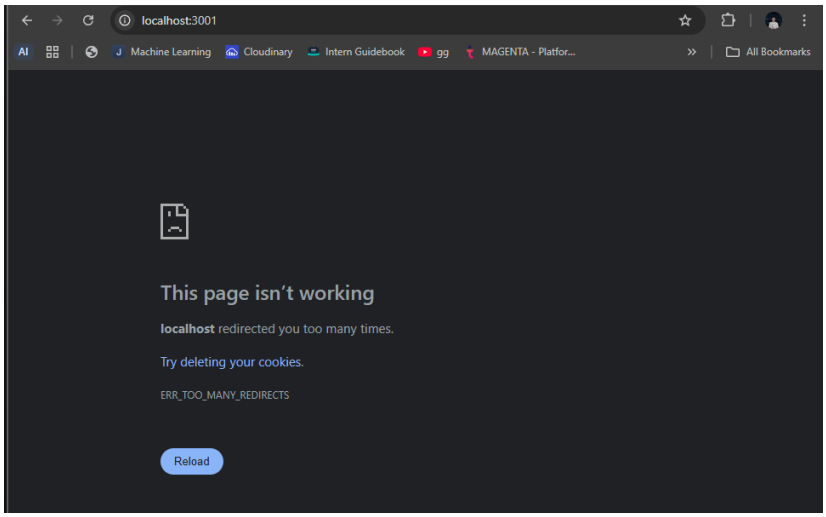
1. Jika menggunakan NextResponse.next() → tidak ada redirect.
2. Jadi masih bisa mengakses ke http://localhost:3000/product



Bagian 3 – Redirect Sederhana

```
index.tsx 3, M ×   middleware.ts U ×
D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-routing\src\pages\product\index.tsx • 3 problems in this file • Modified
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
    return NextResponse.redirect(new URL("/", request.url));
}
```

1. Semua halaman akan redirect ke home dan error dikarenakan terus menerus loading



Bagian 4 – Batasi Route Tertentu

1. Untuk mengatasi pada bagian 3 maka perlu pembatasan route

```
index.tsx 3, M ×   middleware.ts U ×
P2 > Praktikum > next-routing > src > middleware.ts > ...
1   import { NextResponse } from "next/server";
2   import type { NextRequest } from "next/server";
3
4   export function middleware(request: NextRequest) {
5       return NextResponse.redirect(new URL("/", request.url));
6   }
7
8   export const config = {
9       matcher: ["/product", "/about"],
10  };
```

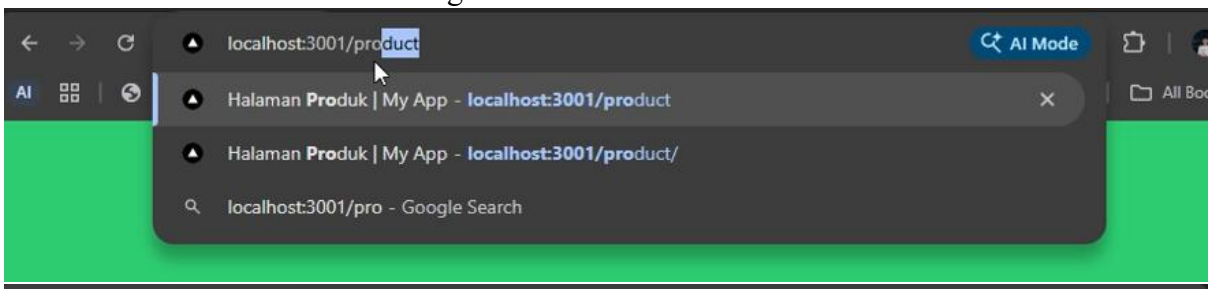
2. Artinya:
   - Middleware hanya berlaku untuk /products dan /about
   - Halaman lain tetap normal
   - Ketika user mengakses halaman produk dan about maka akan langsung redirect ke halaman home

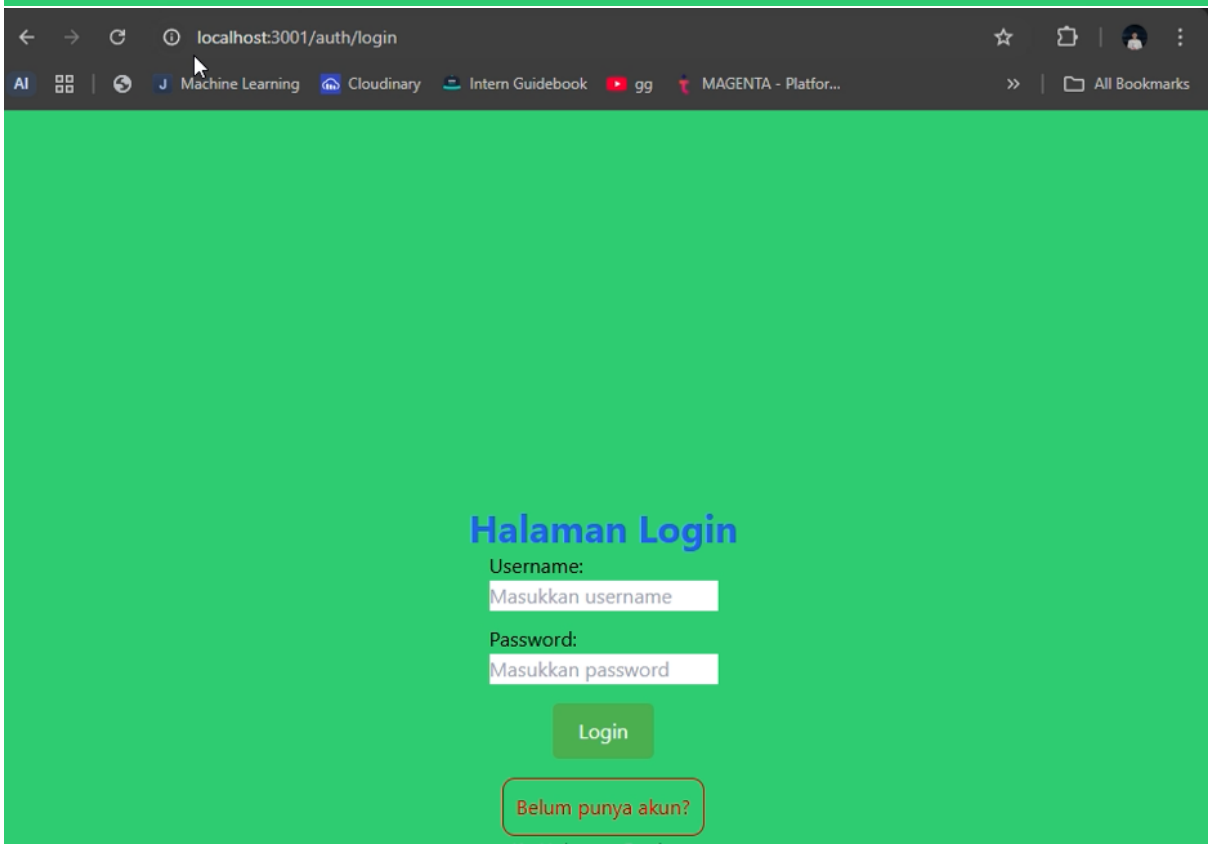
Bagian 5 – Simulasi Sistem Login

1. Modifikasi file middleware.ts

```
index.tsx 3, M   middleware.ts U ×
P2 > Praktikum > next-routing > src > middleware.ts > middleware
1   import { NextResponse } from "next/server";
2   import type { NextRequest } from "next/server";
3
4   export function middleware(request: NextRequest) {
5       const isLogin = false;
6       if (isLogin) {
7           return NextResponse.next();
8       } else {
9           return NextResponse.redirect(new URL("/auth/login", request.url));
10      }
11      // return NextResponse.redirect(new URL("/", request.url));
12  }
13
14  export const config = {
15      matcher: ["/product", "/about"],
16  };
```

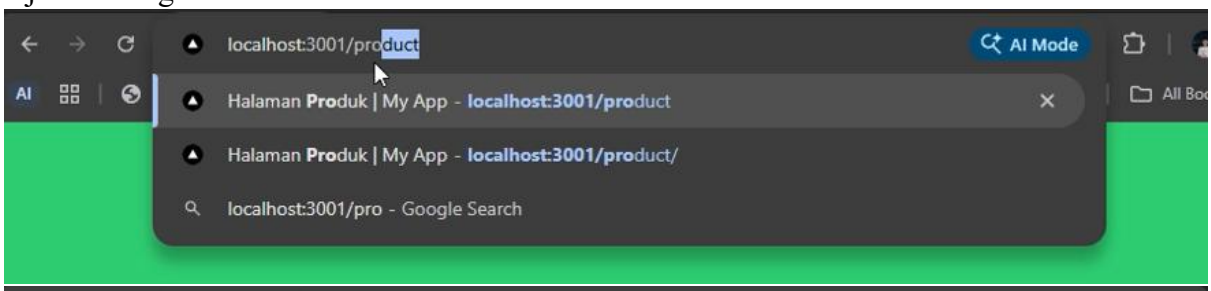
2. Jika user langsung mengakses ke alamat http://localhost:3000/produk tidak akan bisa user akan diarahkan ke halaman login

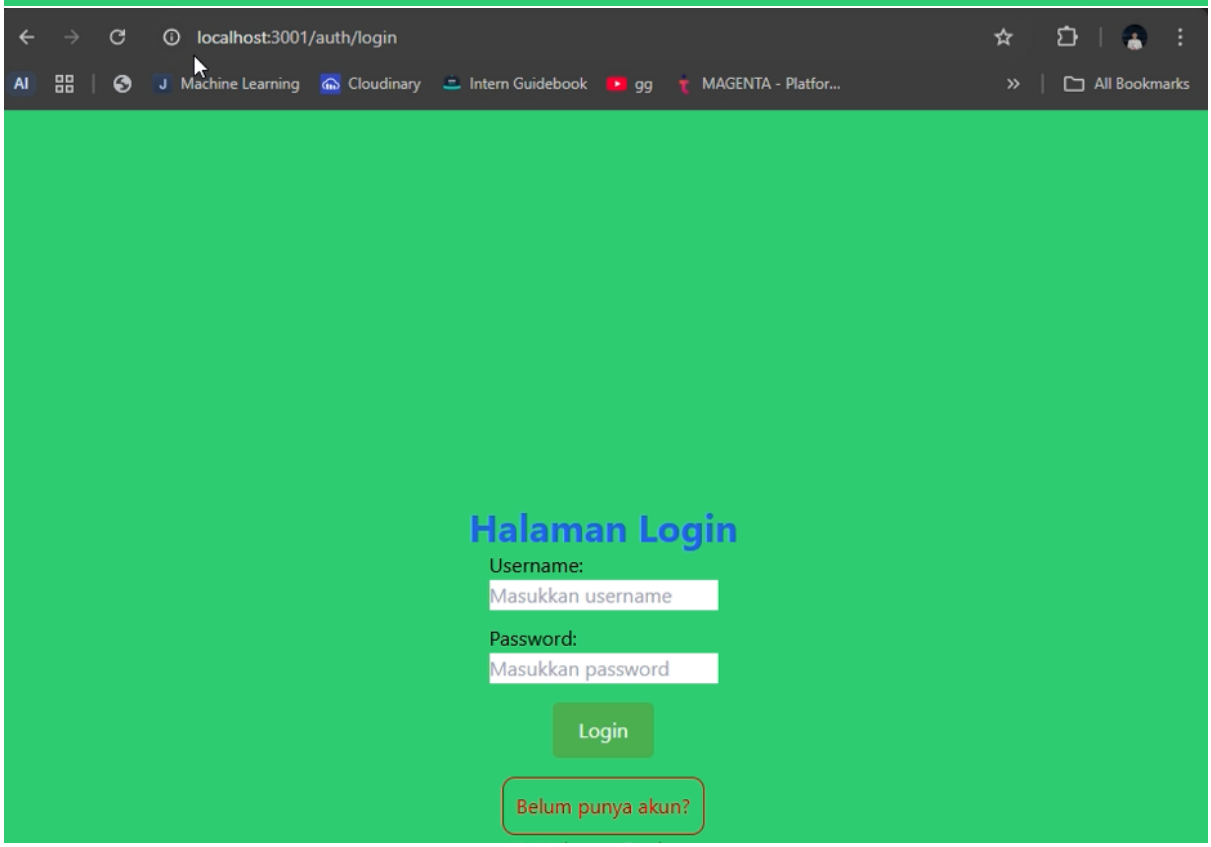




PENGUJIAN

1. Uji 1 – isLogin = false





Hasil: Redirect ke /login

2. Uji 2 – isLogin = true

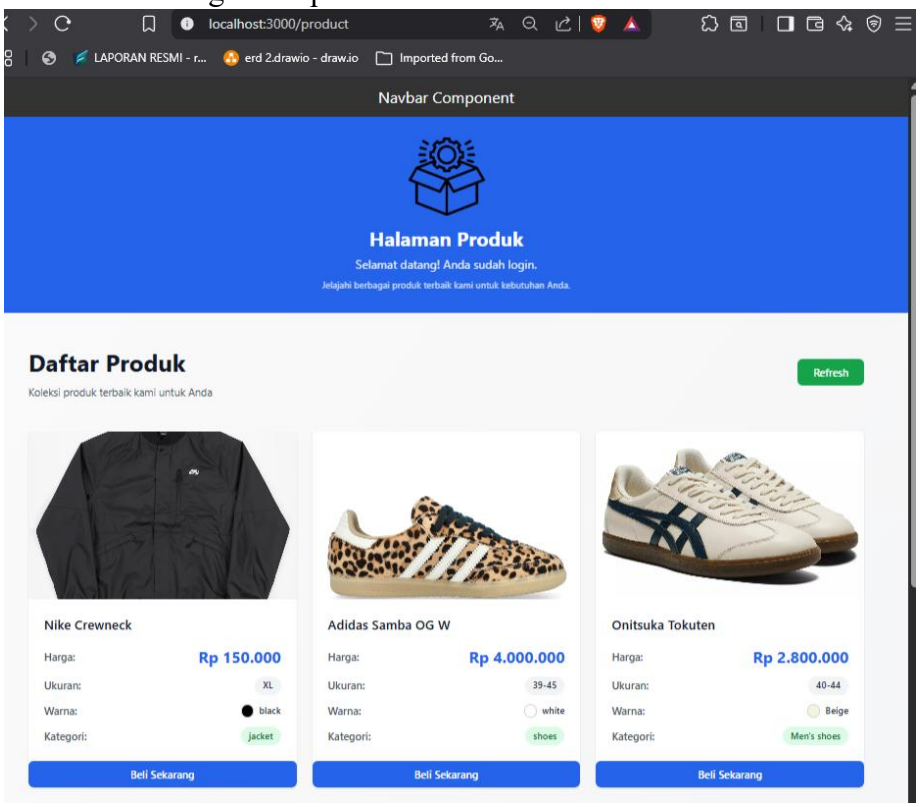
Ubah: const isLogin = true

index.tsx 3, M middleware.ts U

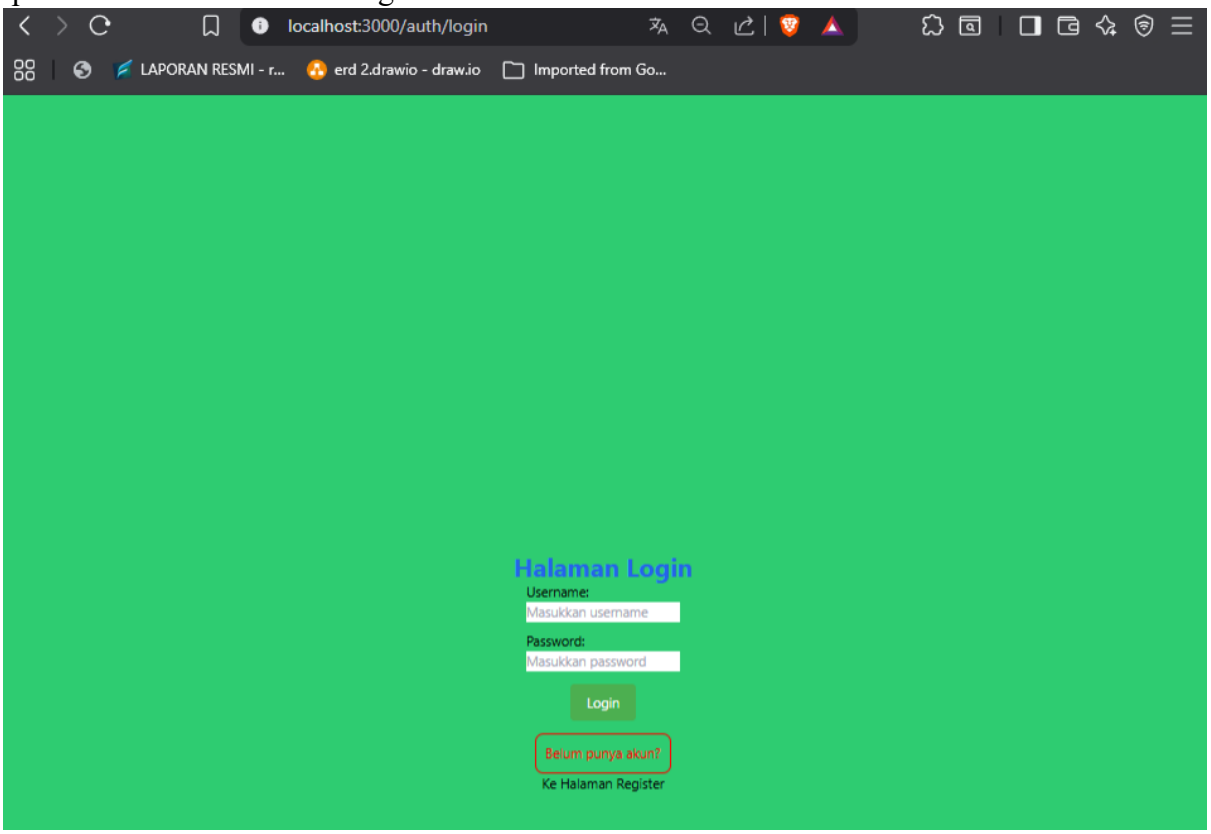
P2 > Praktikum > next-routing > src > middleware.ts > middleware > isLogin

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
    const isLogin = true;
    if (isLogin) {
        return NextResponse.next();
    } else {
        return NextResponse.redirect(new URL("/auth/login", request.url));
    }
    // return NextResponse.redirect(new URL("/", request.url));
}

export const config = {
    matcher: ["/product", "/about"],
};
```

Hasil: Bisa mengakses /products

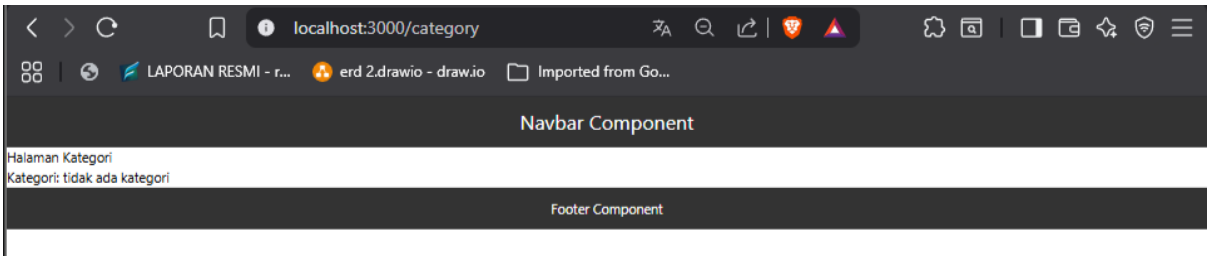


3. Uji 3 – Tambahkan Multiple Route
   export const config = { matcher: ['/products', '/about'] }
   Sekarang:
   - /products dan /about butuh login



Ketika masuk ke http://localhost:3000/product langsung balik ke login

- Halaman lain bebas



TUUGAS PRAKTIKUM

1. Buat halaman:
   - /products
   - /about
   - /login

   (semua sudah dibuat dari awal)
2. Implementasikan Middleware:
   - Redirect ke /login jika belum login.
   - Izinkan akses jika login true

```
P2 > Praktikum > next-routing > src > middleware.ts > ...
1  import { NextResponse } from "next/server";
2  import type { NextRequest } from "next/server";
3
4  export function middleware(request: NextRequest) {
5    const isLogin = request.cookies.get("isLogin")?.value === "true";
6    const { pathname } = request.nextUrl;
7
8    // Jika belum login dan mengakses /product atau /about, redirect ke /login
9    if (!isLogin && (pathname === "/product" || pathname === "/about")) {
10     return NextResponse.redirect(new URL("/auth/login", request.url));
11   }
12
13   // Jika sudah login dan mengakses /auth/login, redirect ke /product
14   if (isLogin && pathname === "/auth/login") {
15     return NextResponse.redirect(new URL("/product", request.url));
16   }
17
18   return NextResponse.next();
19 }
20
21 export const config = {
22   matcher: ["/product", "/about", "/auth/login"],
23 };
```

   - User mengakses /product tanpa login → Middleware redirect ke /login
   - User login dengan username fatih & password 123 → Cookie diset → Redirect ke /product
   - User logout → Cookie & localStorage dihapus → Redirect ke /login

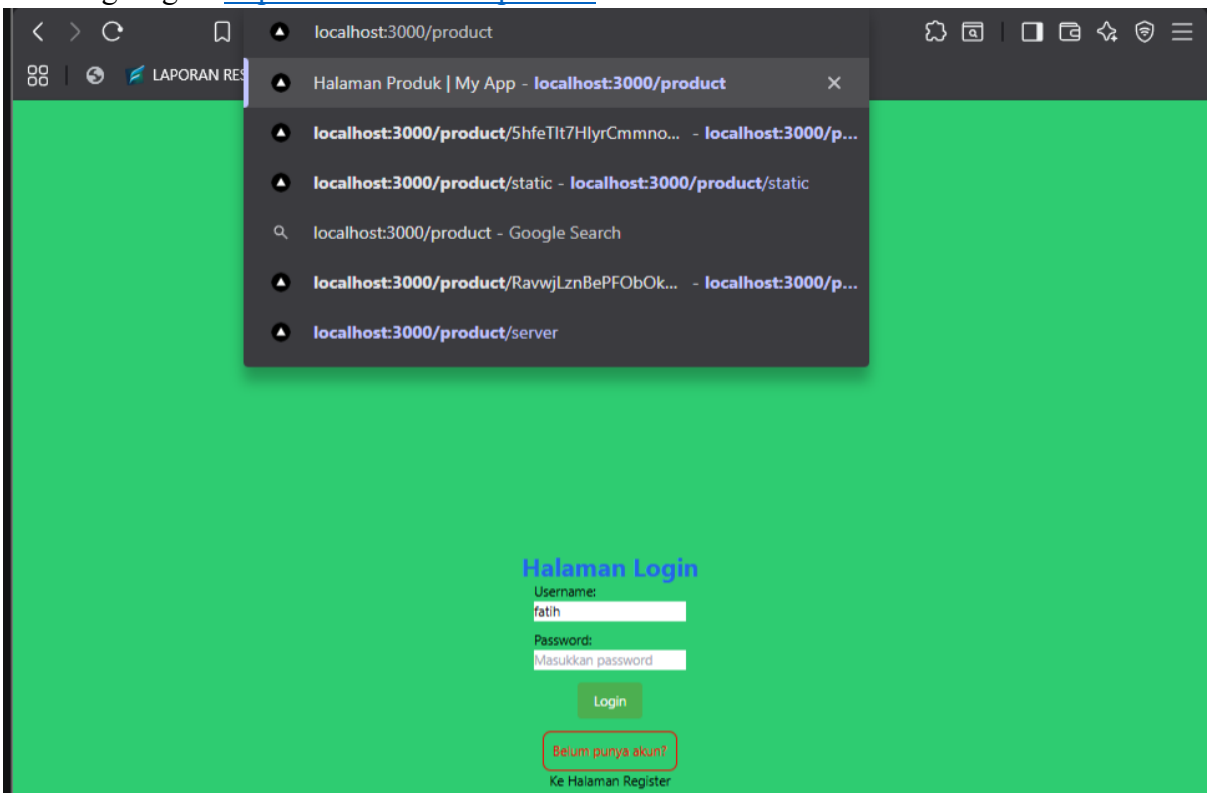
3. Tambahkan proteksi hanya untuk route tertentu.

```
// Route yang memerlukan proteksi autentikasi
const protectedRoutes = ["/product", "/about"];

export function middleware(request: NextRequest) {
  const isLogin = request.cookies.get("isLogin")?.value === "false";
  const { pathname } = request.nextUrl;

  // Cek apakah route saat ini memerlukan proteksi
  const isProtectedRoute = protectedRoutes.some((route) =>
    pathname.startsWith(route),
  );

  // Jika belum login dan mengakses protected route, redirect ke /login
  if (!isLogin && isProtectedRoute) {
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }

  // Jika sudah login dan mengakses /auth/login, redirect ke /product
  if (isLogin && pathname === "/auth/login") {
    return NextResponse.redirect(new URL("/product", request.url));
  }
```

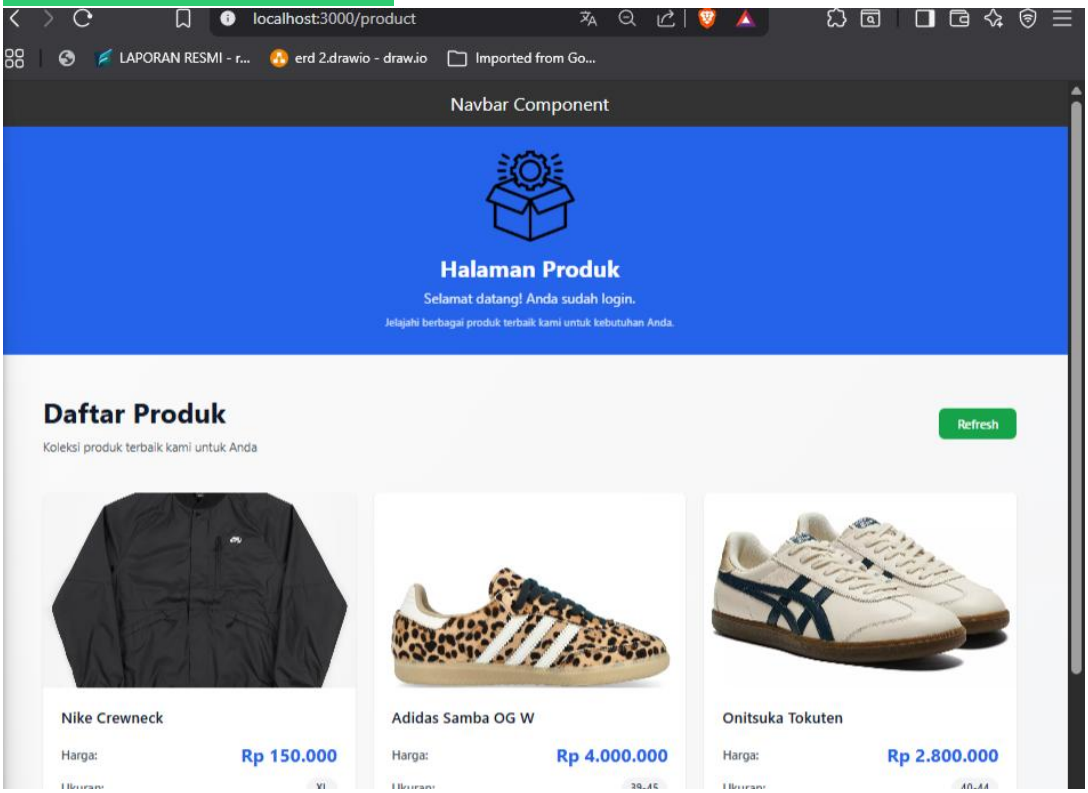
protectedRoutes array: Daftar route yang memerlukan autentikasi

- Menggunakan startsWith() untuk melindungi subroute juga (misal /product/123)

4. Dokumentasikan:
   - Screenshot sebelum dan sesudah redirect.
     Jika langsung ke http://localhost:3000/product



     Maka akan direct ke /auth/login
     Jika sudah login maka akan langsung masuk ke path yang dituju



- Perbandingan dengan useEffect.
  Middleware itu security utama di server, sedangkan useEffect itu cuma validasi tambahan di client. jadi kalau buat login protection, middleware jauh lebih tepat dan aman.

PERTANYAAN ANALISIS

1. **Mengapa middleware lebih aman dibanding useEffect?**
   Middleware berjalan di server (atau edge) sebelum halaman dikirim ke client, sehingga validasi (misalnya auth token) dilakukan lebih awal dan tidak bisa dimanipulasi user. Sedangkan useEffect berjalan di client setelah halaman dirender, jadi data sensitif bisa sempat terlihat sebelum redirect terjadi.

2. **Mengapa middleware tidak menimbulkan glitch?**
   Karena middleware bekerja sebelum rendering, user hanya akan menerima halaman yang sudah valid (misalnya langsung redirect jika belum login). Berbeda dengan useEffect yang bisa menyebabkan "flash" halaman (glitch) karena halaman sempat tampil sebelum diarahkan.

3. **Apa risiko jika semua halaman diproteksi tanpa pengecualian?**
   - Halaman publik (login, register, landing page) jadi tidak bisa diakses
   - Bisa terjadi infinite redirect loop (misalnya halaman login ikut diproteksi)
   - SEO terganggu karena semua halaman butuh autentikasi

   Jadi harus ada pengecualian route tertentu.

4. **Kapan middleware tidak diperlukan?**
   Middleware tidak diperlukan jika:
   - Aplikasi sederhana tanpa autentikasi
   - Validasi cukup dilakukan di client (misalnya UI state saja)
   - Proteksi hanya dibutuhkan di bagian tertentu → bisa pakai guard di component atau API saja
   - Tidak butuh redirect global (cukup handle di masing-masing halaman)

5. **Apa perbedaan middleware dan API route?**
   Middleware:
   - Jalan sebelum request diproses
   - Digunakan untuk auth, redirect, logging, dll
   - Tidak mengembalikan data utama, tapi memodifikasi request/response

   API Route:
   - Endpoint backend untuk proses data (CRUD, auth, dll)
   - Mengembalikan response (JSON, dll)
   - Dipanggil oleh client (fetch/AJAX)